

Plano de Trabalho — Jogo Roguelike em PHP

2026-09-18 · @Someone

Visão geral

O projeto é um jogo roguelike de sobrevivência com ataque automático, no molde de Vampire Survivors, com ambientação inspirada em Minecraft. O jogador controla apenas a movimentação; as armas atacam sozinhas conforme seus cooldowns, enquanto ondas progressivamente mais fortes de inimigos surgem no mapa.

A partida consiste em sobreviver, acumular experiência, evoluir armas e itens, derrotar o boss e registrar a maior pontuação possível no ranking.

Item	Definição
Equipe	4 ou mais integrantes
Prazo	1 a 2 meses (planejado em 8 semanas)
Linguagem	PHP, exclusivamente
Interface	Janela gráfica; terminal não permitido
Entrega	Jogo executável, código-fonte e apresentação

Escopo obrigatório

Os requisitos do documento da disciplina, agrupados por sistema. A coluna Depende de indica qual sistema precisa existir antes.

Sistema	Requisitos	Conteúdo mínimo	Depende de
Menus e navegação	RF01, RF02	Menu inicial, seleção de personagem, instruções	—
Loop e movimentação	RF03, RF04	Mapa, posicionamento, controle por teclado	—
Inimigos	RF05, RF22	3 tipos comuns e 1 boss, com atributos próprios	Loop
Armas e combate	RF06, RF07, RF08, RF18, RF20	4 armas com mira e cooldown próprios	Loop, Inimigos

Sistema	Requisitos	Conteúdo mínimo	Depende de
Experiência e nível	RF11, RF12, RF13	Drop de XP, acúmulo, fórmula de evolução	Inimigos
Level up	RF14, RF15, RF19, RF21	Pausa, 3 opções, limite de 2 armas	XP, Armas
Personagens	RF16, RF17	Steve e Alex, atributos e passivas distintas	Loop
Fim de jogo	RF09	Derrota por vida zero, vitória a definir	Combate
Baús e cura	RF26, RF27	Baús por raridade após o boss, item de cura	Boss
Dificuldade	RF28, RF29, RF30	Ondas, escalonamento, cronômetro	Inimigos
Pontuação e ranking	RF32, RF33, RF34	Cálculo, cadastro de nome, ranking ordenado	Banco de dados

Os requisitos não funcionais atravessam todos os sistemas: banco relacional (RNF01), arquitetura modular (RNF02), feedback visual e sonoro (RNF05), desempenho estável com muitos inimigos (RNF06) e testes unitários (RNF03, opcional).

O documento original tem inconsistências que devem ser confirmadas com o professor: o RF09 menciona navios afundados, herdado de outro enunciado; existem dois requisitos numerados RF22; e faltam os números RF10, RF23 a RF25 e RF31.

Requisitos adicionais escolhidos

O enunciado pede no mínimo 2 requisitos adicionais. A proposta é implementar 3, escolhidos por serem lógica pura, sem custo de renderização adicional.

Requisito	O que faz	Justificativa
RF39 — Passivas	Steve ganha dano a cada N níveis; Alex reduz cooldown em 0,2s a cada 4 níveis	Já exigido de fato pelo RF02, que pede habilidades passivas próprias
RF35 — Eventos aleatórios	Horda e Chuva de XP	Reaproveita o sistema de ondas do RF29, custo baixo
RF36 — Inimigos especiais	Slime que se divide ao morrer e um inimigo à distância	Reaproveita o sistema de spawn e dá variedade visível

O RF37 (evoluções especiais) e o RF38 (drops de cenário) ficam como escopo opcional, a implementar apenas se houver folga após a semana 6.

Decisões de projeto

O enunciado deixa quatro pontos a critério do grupo. As decisões abaixo ficam registradas para evitar retrabalho.

Condição de vitória (RF09). Derrotar o boss, que surge aos 10 minutos de sobrevivência. A partida continua após a vitória em modo livre, até a morte do personagem, para que o jogador possa aumentar a pontuação.

Fórmula de pontuação (RF32). Combina os três fatores sugeridos pelo enunciado:

$$P = (N \times 100) + (I \times 10) + (T \times 2) + (V \times 500)$$

Onde N é o nível alcançado, I os inimigos derrotados, T o tempo em segundos e V vale 1 em caso de vitória e 0 caso contrário.

Critério de desempate (RF34). Mesma pontuação: vence quem sobreviveu menos tempo, premiando eficiência. Persistindo o empate, vence a partida mais antiga.

Fórmula de nível (RF13). A sugerida pelo enunciado, em três faixas: níveis 0 a 15 usam $2n+7$; níveis 16 a 30 usam $5n-38$; níveis 31 em diante usam $9n-158$.

Tecnologias e bibliotecas

Todo o código do projeto é escrito em PHP. As bibliotecas abaixo são consumidas a partir do PHP, nenhuma exige escrever ou compilar código em outra linguagem.

Camada	Escolha	Justificativa
Linguagem	PHP 8.3 ou superior	Exigência da disciplina; FFI requer 8.1+
Renderização	Raylib, via FFI	PHP não tem recurso nativo de janela; Raylib traz imagem, fonte, áudio e colisão num arquivo só
Banco de dados	SQLite 3	Arquivo local, sem servidor a instalar; atende o RNF01
Acesso a dados	PDO nativo	Sem ORM: o SQL fica visível, que é o que o RNF02 pede demonstrar
Autoload	Composer, PSR-4	Organiza o MVC sem includes manuais
Testes	PHPUnit	Cobre o RNF03
Configuração	Arquivos JSON	Atributos de armas, inimigos e personagens fora do código, para balanceamento rápido

Sobre o Raylib e a regra de linguagem única. O `libraylib` é um binário já compilado; o FFI é um recurso oficial do PHP que permite chamá-lo. A mesma estrutura já existe no acesso ao banco: o PDO conversa com a `libsqlite3`, também escrita em C. Como o PHP não oferece nenhuma forma nativa de abrir janela, qualquer interface gráfica passa necessariamente por uma biblioteca de sistema.

Ferramentas de apoio. Figma para desenhar as telas antes de programar; Piskel ou LibreSprite para os sprites; jfxr ou Bfxr para efeitos sonoros; Audacity para edição de áudio. Formatos: PNG para imagem, WAV para efeitos curtos, OGG para música.

Assets. Não usar arquivos originais do Minecraft. Usar sprites próprios ou pacotes de domínio público, como os do Kenney.

Desempenho (RNF06). Habilitar OPcache e JIT no CLI, via `php.ini` :
`opcache.enable_cli=1`, `opcache.jit_buffer_size=64M` e `opcache.jit=tracing`. O ganho incide justamente sobre laços aritméticos, que é o perfil do loop de colisão.

Qualidade (opcional). PHPStan para análise estática e Laravel Pint para formatação. Nenhum dos dois entra no jogo em execução.

Arquitetura

O RNF02 pede código modular com separação de responsabilidades. O padrão adotado é MVC, com uma regra central: o Model não conhece a View. Nenhuma classe de regra de jogo importa Raylib ou chama função de desenho.

Essa regra tem duas consequências práticas. O Model pode ser testado com PHPUnit sem abrir janela, e duas frentes da equipe trabalham em paralelo desde o primeiro dia.

```

flowchart LR
    Input[Entrada<br/>teclado] --> Controller
    Controller --> Model[Model<br/>regras do jogo]
    Model --> View[View<br/>Raylib]
    Model --> Repo[Repository<br/>PDO]
    Repo --> DB[(SQLite)]

```

A renderização fica atrás de uma interface `RendererInterface`, com os métodos de desenhar entidade, desenhar HUD e apresentar quadro. Se o Raylib apresentar problema em alguma máquina, basta escrever outra implementação sem tocar no Model.

Estrutura de pastas

jogo.php	ponto de entrada
composer.json	
lib/	libraylib (fora do Git)
config/	JSON de armas, inimigos, personagens
assets/	sprites, sons, fontes
database/	
schema.sql	
ranking.sqlite	gerado, fora do Git
src/	
Core/	GameLoop, Timer, InputHandler, Ffi
Model/	
Entity/	Player, Enemy, Boss, Projectile, Chest
Weapon/	Weapon, Sword, Bow, cooldown e mira
System/	XpSystem, WaveManager, DamageSystem, ScoreCalculator
View/	RendererInterface, RaylibRenderer, Hud, MenuView,
LevelUpView	
Controller/	MenuController, MatchController, RankingController
Repository/	Connection, ScoreRepository
tests/	PHPUnit

O `.gitignore` deve excluir `vendor/`, `lib/` e `database/ranking.sqlite`. O `schema.sql` é versionado no lugar do banco, para evitar conflito de merge em arquivo binário.

Divisão de tarefas

Quatro frentes, uma por integrante. Grupos maiores dividem a frente de conteúdo, que é a mais volumosa.

Frente	Responsabilidades	Requisitos	Entrega isolada
A — Núcleo e combate	GameLoop, movimentação, colisão, dano, armas e cooldown	RF03 a RF08, RF18, RF20	Testável sem janela
B — Progressão	XP, níveis, level up, upgrades, itens passivos, personagens e passivas	RF11 a RF17, RF19, RF21, RF39	Testável sem janela
C — Conteúdo e dificuldade	Inimigos, boss, ondas, escalonamento, baús, eventos, inimigos especiais	RF05, RF22, RF26 a RF30, RF35, RF36	Testável sem janela
D — Interface e dados	Renderização, menus, HUD, som, banco, pontuação, ranking	RF01, RF02, RF09, RF32 a RF34, RNF01, RNF05	Depende do Model

As frentes A, B e C produzem código de Model, sem dependência de Raylib, e podem começar no primeiro dia. A frente D começa pelo banco e pelos menus, que também independem do resto.

Contratos a definir na semana 1. Antes de qualquer código, as quatro frentes precisam concordar sobre as assinaturas compartilhadas: a interface `RendererInterface`, a classe `Entity` base, o formato dos JSON de configuração e a assinatura do `ScoreRepository`. Divergência aqui é a principal causa de retrabalho em trabalho de grupo.

Cronograma

Oito semanas, com a última reservada para folga. O marco de cada semana é algo demonstrável, não uma porcentagem.

Semana	Marco	Responsáveis
1	Ambiente validado: Raylib abre janela em todas as máquinas, repositório e contratos definidos	Todos
2	Jogador se move na tela; banco criado e ranking gravando	A, D
3	Inimigos perseguem, colidem e morrem; XP dropa e acumula	A, B, C

Semana	Marco	Responsáveis
4	Duas armas atacando automaticamente; tela de level up funcionando	A, B
5	Quatro armas, upgrades, itens passivos, dois personagens com passivas	A, B
6	Ondas, escalonamento, boss e baús; partida jogável de ponta a ponta	C, D
7	Som, feedback visual, requisitos adicionais, testes unitários	Todos
8	Folga, correção de bugs, ajuste de balanceamento e ensaio da apresentação	Todos

O marco da semana 6 é o mais importante: a partir dele existe um jogo completo e entregável. Tudo depois disso é acabamento, e pode ser cortado sem comprometer a entrega.

Regra de corte. Se a semana 6 chegar sem a partida completa, os requisitos adicionais opcionais (RF37 e RF38) saem primeiro, depois o polimento sonoro, depois os testes unitários, que são opcionais no enunciado.

Riscos e pendências

A confirmar com o professor. Quatro pontos podem alterar o plano e devem ser resolvidos antes da semana 2.

- ☐ O uso do Raylib via FFI atende à regra de linguagem única? Todo o código do grupo é PHP; nenhuma linha de C é escrita ou compilada, e o mesmo mecanismo já sustenta o acesso ao SQLite exigido pelo RNF01.
- ☐ Bibliotecas externas via Composer são permitidas?
- ☐ O RF09 menciona navios afundados, herdado de outro enunciado. A condição de vitória proposta é derrotar o boss.
- ☐ Faltam os números RF10, RF23 a RF25 e RF31, e há dois RF22. Algum requisito ficou de fora do documento?

Riscos técnicos

Risco	Probabilidade	Mitigação
FFI ou Raylib não funcionar em alguma máquina	Média	Validar na semana 1, antes de qualquer código; SDL como alternativa
Desempenho insuficiente com muitos inimigos	Média	OPcache com JIT; limitar inimigos simultâneos; medir cedo
Segmentation fault por assinatura FFI incorreta	Baixa	Conferir tipos contra o <code>raylib.h</code> da versão baixada
Divergência de versão do PHP no grupo	Alta se não tratada	Fixar 8.3 na semana 1 e registrar no README
Integração tardia entre as frentes	Alta	Contratos definidos na semana 1; integração semanal, não no fim

O maior risco não é técnico e sim de cronograma: as frentes trabalharem isoladas por seis semanas e só integrarem no fim. A integração deve acontecer toda semana, mesmo incompleta.